

OpenCode

Ejecutar Comandos

Uso de `!`

Permite ejecutar comandos directamente desde el chat del agente sin gastar tokens, ya que la salida no pasa por el modelo de lenguaje.

Cambiar Funcionalidad

Modos Plan y Build

- En OpenCode existen dos modos de trabajo: **Plan**, en el que el agente únicamente analiza y responde sin ejecutar acciones, y **Build**, en el que el agente sí ejecuta comandos sobre el proyecto.
 - Se alterna entre ambos modos con la tecla `TAB`.
-

Búsqueda específica

Uso de `@`

Permite referenciar archivos puntuales dentro del chat, ayudando al agente a ubicarlos rápidamente y evitando que gaste tokens en búsquedas innecesarias por todo el proyecto.

Búsquedas `/`

Uso de `/`

Abre un menú de acciones rápidas que permite cambiar de agente, modificar configuraciones del agente activo, y ejecutar comandos disponibles, entre otras opciones.

Ctrl + Shift + P

Paleta de comandos

Abre la paleta de comandos del editor, dando acceso directo a todas las acciones y opciones disponibles en OpenCode (similar al uso de `/` , pero a nivel general del editor, no solo del chat).

El archivo `agents.md`

Qué es

Es un archivo en Markdown que funciona como manual de instrucciones para la IA, definiendo las reglas de un proyecto completo (no de una tarea puntual, como sí lo hace un *skill*). Es un estándar abierto: lo usan OpenCode, Cursor, Cline y VS Code, entre otros.

Para qué sirve

Evita que la IA pierda contexto, alucine o tome decisiones que contradigan las preferencias del desarrollador, y elimina la necesidad de repetir instrucciones básicas en cada prompt. Aun así, una instrucción directa del usuario siempre tiene prioridad sobre lo escrito en el archivo.

Qué debe incluir

Debe ser breve para no consumir tokens de más. Suele contener decisiones de arquitectura (por ejemplo, usar Vanilla JS en vez de un framework), gestión de dependencias (qué librerías están permitidas o prohibidas, qué gestor de paquetes usar), comandos preferidos (cómo levantar el servidor) y preferencias generales de estilo de código.

Cómo se crea

En OpenCode se genera con el comando `/init` . Lo ideal es ejecutarlo al inicio del proyecto, para fijar las reglas desde el principio y evitar que la IA decida por su cuenta. Sin embargo, también puede crearse o editarse en cualquier etapa si el desarrollador nota que la IA se está desviando de lo esperado.

Cómo se mantiene

El archivo no es estático: es una guía viva que el desarrollador edita manualmente a medida que cambian sus decisiones técnicas. No necesita estar "terminado"; antes de pedir una tarea compleja conviene actualizarlo con las instrucciones clave para esa tarea, ya que la IA siempre lee la versión más reciente.

Skills

Qué son

Son un estándar que permite ampliar la experiencia de los agentes, convirtiéndolos en "expertos" en una tarea, librería o tecnología específica.

Origen de las skills

Pueden crearse skills propias, formando una biblioteca personalizada que enseña a la IA cómo realizar ciertas tareas o usar determinadas librerías. También existe `skills.sh`, un sitio con skills ya creadas, auditadas y descargables sin riesgo, a diferencia de descargar directamente desde repositorios sin revisar en GitHub.

Autoskills

Comando: `npx autoskills@latest`. Esta herramienta detecta automáticamente las dependencias del proyecto (librerías, frameworks, lenguajes, etc.), busca en internet y GitHub, y descarga las mejores skills disponibles para esas dependencias.

Agentes

Qué son

Los agentes no son los modelos (como ChatGPT, DeepSeek, etc.), sino los distintos modos de trabajo sobre el proyecto, como Build y Plan en OpenCode.

Subagentes

Son agentes que permiten ejecutar nuevas operaciones sobre el proyecto sin detener el hilo principal de la conversación. Cuando hay varios agentes trabajando sobre la misma sesión o proyecto, se puede navegar entre ellos con las flechas izquierda y derecha. Algunos ejemplos son:

- `@explore`
- `@general`

Crear agentes

Se definen en archivos `.md` dentro de la carpeta `.opencode/agents`, con un encabezado de configuración seguido del *system prompt* del agente. El encabezado suele incluir:

- **Modelo:** qué modelo de IA usará ese agente.
- **Temperatura:** baja, para respuestas más precisas y menos creativas.
- **Permisos:** qué puede y qué no puede hacer (por ejemplo, edición de archivos denegada, o permisos selectivos de `bash` como permitir `git status`, `git diff` y `git log`, pidiendo

confirmación para el resto).

Luego, en el cuerpo del archivo se describe el rol del agente, como "Eres un revisor de código pragmático", indicando que debe enfocarse en regresiones, problemas de seguridad y tests, sin señalar gustos de estilo salvo que afecten al funcionamiento o comportamiento del código. Siguiendo este mismo formato se puede crear, por ejemplo, un agente `reviewer.md` y un agente `security.md`.

Comandos

- `/status` : muestra la cantidad de tokens usados y el límite disponible.
- `/timeline` : muestra un resumen de cada petición realizada y un breve extracto de las respuestas del agente.
- `/themes` : permite cambiar el tema visual (por ejemplo, *synthwave*).
- `/new` : crea una nueva sesión, las cuales pueden consultarse luego con `Ctrl+Shift+P` o mediante `/commands`.
- `/compact` : compacta la sesión, generando un resumen y eliminando el historial anterior; ese resumen pasa a ser el nuevo contexto.

Dudas frecuentes

El historial de la sesión forma parte del contexto del agente. Los comandos ejecutados en la terminal (shell), junto con sus respuestas, son reconocidos por el agente tanto en modo Build como en modo Plan.

- **Mejores modelos gratuitos:** MiniMax 2.5 y Big Pickle.
- **Mejores modelos de pago:** GLM (gran cantidad de tokens, económico y con buena capacidad de razonamiento), MiniMax 2.7 y DeepSeek V4 Pro.